

Manual de laboratorio: streaming concurrente con mutex y variables condicionales

Práctica 4: Planificación de reproducción de segmentos HLS (.m3u8) con prioridades

Nota: el escenario está inspirado en cómo los reproductores reales gestionan colas, buffers y recursos compartidos.

PRÁCTICA FUNCIONAL EN: **C**

ROL Auditor de concurrencia / Ingeniero de infraestructura de streaming.

MISIÓN Reproducir segmentos HLS desde un manifiesto .m3u8 manteniendo prioridad y capacidad máxima simultánea.

REQUISITO Prohibido el uso de interfaces gráficas. Las evidencias deberán obtenerse desde terminal.

SE DEBE ENTREGAR

- Código compilable y funcional, con nombres, apellidos y correos, que incluya las respuestas a las cuestiones planteadas (numeradas).

1. FASE 1: Contexto técnico (streaming HLS y manifiestos .m3u8)

1.1 ¿Qué es un .m3u8 y por qué existe?

Un archivo .m3u8 es un **manifiesto de reproducción** común en HLS (*HTTP Live Streaming*). Su función es describir qué fragmentos de vídeo (*segments*) deben descargarse y reproducirse, normalmente en formato .ts o .m4s. El manifiesto actúa como un *mapa de navegación* para el reproductor:

- Contiene la lista de URL o rutas de segmentos.
- Permite reproducción progresiva: se empieza a reproducir mientras se descargan segmentos posteriores.
- Facilita la conmutación de calidad (bitrate/resolución) en sistemas adaptativos.

1.2 ¿Dónde entra la sincronización?

Un reproductor real no puede ejecutar reproducciones infinitas en paralelo: está limitado por **CPU, decodificadores, buffer y ancho de banda**. Por ello, necesita un control de concurrencia que garantice:

- No exceder una capacidad máxima de reproducciones simultáneas.
- Ordenar solicitudes en cola cuando el recurso está saturado.
- Aplicar prioridad de servicio cuando existen clases de usuarios diferentes.

MODELO MENTAL Imagine la capacidad de decodificación como una sala con plazas limitadas. Si está llena, toca esperar turno.

2. FASE 2: Especificación del sistema (colas + mutex + variables de condición)

En esta práctica implementará un simulador de reproducción concurrente con estas reglas:

2.1 Clases de servicio

Existen dos clases de solicitudes (dos colas):

- **PRIORITY**: solicitudes con precedencia absoluta.
- **ECONOMY**: solicitudes de menor precedencia (esperan si hay solicitudes **PRIORITY** pendientes).

2.2 Recurso crítico y capacidad

El sistema tiene una capacidad global de reproducción simultánea:

$$\text{ACTIVE} < \text{CAPACITY}$$

- **CAPACITY** es el máximo de segmentos que pueden reproducirse en paralelo.
- **ACTIVE** es el contador de segmentos reproduciéndose en este instante.

2.3 Reglas de entrada al recurso

1. Si $\text{ACTIVE} == \text{CAPACITY}$, cualquier solicitud debe esperar.
2. Si una solicitud es **ECONOMY** y existe al menos una solicitud **PRIORITY** esperando, entonces **ECONOMY** también debe esperar, aunque haya hueco libre.
3. Dentro de cada clase se respetará el orden de llegada mediante un modelo de *tickets*. Esto implica que, cuando una URL entra en cola, se le asigna un número y las solicitudes se procesarán por orden FIFO dentro de su clase.

MECÁNICA OBLIGATORIA Debe usarse obligatoriamente: 1 mutex + 2 variables condicionales (una por cola) + contadores de espera + control de capacidad + tickets por clase.

La práctica debe completarse a partir del fichero base incompleto proporcionado. No se aceptarán soluciones rehechas con otra estructura ni con otros mecanismos de sincronización.

3. FASE 3: Implementación (hilos + mutex + variables de condición)

3.1 Entrada: manifiesto de reproducción

El manifiesto `.m3u8` entregado es ficticio: contiene URL falsas y, en cada línea válida, se indica la clase de servicio y el segmento.

Ejemplo conceptual de línea:

Listing 1: Formato conceptual de una línea del manifiesto

```
1 | PRIORITY https://cdn.fake/stream/seg_0001.ts
2 | ECONOMY  https://cdn.fake/stream/seg_0002.ts
```

3.2 Modelo de ejecución

Cada línea del manifiesto representa una solicitud que se implementa como:

- Un **hilo** que intenta entrar al reproductor.
- Si entra, simula descarga y reproducción mediante `sleep`.
- Al finalizar, libera un hueco y despierta a la cola correspondiente.

3.3 Mensajes obligatorios

Cada hilo debe imprimir trazas claras:

- Mensaje al comenzar, indicando identificador de hilo, clase y URL.
- Mensaje al terminar que incluya exactamente la cadena “Archivo descargado y reproducido correctamente”.
- En cualquier ejecución, si existen solicitudes **PRIORITY** esperando, ninguna solicitud **ECONOMY** puede iniciar antes. Es decir, las peticiones **PRIORITY** tienen prioridad absoluta sobre las **ECONOMY**.

4. FASE 4: Cuestiones y entrega

1 - Se debe entregar un programa completo, compilable y funcional, construido a partir del fichero base incompleto proporcionado. No se aceptará rehacer la práctica con una estructura distinta.

2 - Identifique las variables compartidas del sistema y justifique por qué deben protegerse con un mutex. Debe incluir esta explicación en comentarios dentro del código.

3 - Explique la diferencia entre bloqueo activo y uso de variables condicionales para dormir hilos. Debe incluir esta explicación en comentarios dentro del código.

4 - Describa su implementación del modelo de tickets para respetar el orden dentro de cada clase. Debe incluir esta explicación en comentarios dentro del código.

5 - Indique la condición exacta que debe cumplirse para que una solicitud ECONOMY pueda comenzar a reproducir. Debe incluir esta explicación en comentarios dentro del código.

6 - Analice si podría ocurrir inanición de ECONOMY si llegan continuamente solicitudes PRIORITY. Indique cómo lo mitigaría un sistema real. Debe incluir esta explicación en comentarios dentro del código.